
BrainScan AI

Rapport de Projet

Détection et classification de tumeurs cérébrales par IRM

ResNet50 · Transfer Learning · Grad-CAM
98.96% accuracy · 22 analyses modulaires · Production-ready

Modèle : ResNet50 (Transfer Learning ImageNet)
Précision test : **98.96%** sur 1 054 images
Stack : PyTorch · FastAPI · Vanilla JS · Docker
Tests : 17/17 passing · CI GitHub Actions
Documentation : 17 READMEs + 17 FLOW.md

4 mai 2026

Avertissement médical

Cet outil est destiné à un **usage éducatif et de recherche uniquement**. Il ne remplace en aucun cas un diagnostic médical professionnel. Consulter toujours un professionnel de santé qualifié.

Résumé exécutif

BrainScan AI est un système complet d'intelligence artificielle médicale qui classe automatiquement les IRM cérébrales en quatre catégories : **Glioma**, « **Méningiome**, **Aucune tumeur** et **Tumeur hypophysaire**.

Le projet couvre l'**intégralité du cycle de vie d'un produit ML** : préparation du dataset Kaggle (7 023 images), entraînement par transfer learning en deux stages d'un ResNet50, évaluation rigoureuse sur 1 054 images de test, génération automatique de 22 figures statistiques, et déploiement via une API REST FastAPI accompagnée d'une interface web Vanilla JS premium et d'une alternative Streamlit.

Performances obtenues

- **Test accuracy** : 98.96% (1 043/1 054 corrects)
- **Inférence CPU** : ~36 ms/image · **GPU** : ~10 ms/image
- **Grad-CAM** en temps réel via hooks PyTorch (interprétabilité)
- **Tests** : 17/17 passing en moins de 10 secondes
- **Architecture modulaire** : 22 analyses auto-découvertes, sans modification du code central

Le code suit les meilleures pratiques modernes : containerisation Docker multi-stage, pipeline CI/CD GitHub Actions (lint → test → build), pre-commit hooks, configuration via variables d'environnement `.env`, rate limiting et authentification optionnelle par clé API. La documentation est exhaustive avec 17 READMEs détaillés et 17 fichiers FLOW.md offrant des arbres de décision visuels pour chaque composant.

Table des matières

Résumé exécutif	1
1 Introduction	5
1.1 Contexte médical	5
1.2 Problématique	5
1.3 Objectifs du projet	5
1.4 Stack technique global	6
2 Architecture globale du projet	7
2.1 Vue d'ensemble	7
2.2 Pipeline complet en 6 phases	8
2.3 Principes de design	8
3 Phase 1 : Préparation des données	9
3.1 Source du dataset	9
3.2 Pipeline de prétraitement	9
3.3 Augmentation des données (Albumentations)	9
3.4 Split stratifié	10
4 Phase 2 : Entraînement du modèle	11
4.1 Architecture du modèle	11
4.2 Stratégie de transfer learning en 2 stages	12
4.2.1 Pourquoi 2 stages ?	12
4.3 Callbacks et optimisations	12
4.4 Hyperparamètres complets	13
4.5 Durée d'entraînement	13
5 Phase 3 : Évaluation	14
5.1 Procédure d'évaluation	14
5.2 Résultats globaux	14
5.3 Métriques par classe	14
5.4 Matrice de confusion	15
5.5 Performances temporelles	15
5.6 Sorties d'évaluation	15
6 Phase 4 : Statistiques modulaires (22 analyses)	16
6.1 Architecture par registre	16
6.2 Structure du package	16
6.3 Catalogue des 22 analyses	16
6.3.1 Vue d'ensemble (overview) — 2 analyses	16
6.3.2 Propriétés des images (image) — 9 analyses	17

6.3.3	Entraînement (training) — 2 analyses	17
6.3.4	Évaluation (evaluation) — 5 analyses	17
6.3.5	Analyse des erreurs (errors) — 4 analyses	17
6.4	Mécanisme d'auto-découverte	17
6.5	Organisation automatique des sorties	18
6.6	CLI flexible	18
7	Phase 5 : Déploiement	20
7.1	API FastAPI	20
7.1.1	Architecture	20
7.1.2	Les 7 endpoints	20
7.1.3	Pipeline d'une requête POST /predict	20
7.1.4	Grad-CAM en pur PyTorch	21
7.1.5	Sécurité	21
7.2	Frontend Vanilla JavaScript	21
7.2.1	Stack technique	21
7.2.2	Architecture en 16 blocs	22
7.2.3	Fonctionnalités utilisateur	22
7.3	Frontend alternatif : Streamlit	23
7.4	Containerisation Docker	23
7.4.1	Dockerfile multi-stage	23
7.4.2	docker-compose	24
8	Phase 6 : Qualité du code	25
8.1	Suite de tests pytest	25
8.1.1	Détail des 13 tests API	25
8.1.2	Fixtures partagées (confest.py)	25
8.2	CI/CD GitHub Actions	26
8.3	Pre-commit hooks	26
8.4	Configuration centralisée (pyproject.toml)	26
9	Résultats et benchmarks	28
9.1	Synthèse des performances	28
9.2	Comparaison avant/après refactor	28
10	Conclusion et perspectives	30
10.1	Bilan	30
10.2	Limites actuelles	30
10.3	Pistes d'amélioration	30
10.3.1	Performance modèle	30
10.3.2	Production	31
10.3.3	Fonctionnalités	31
10.4	Mot final	31

A	Annexes	32
A.1	Inventaire complet des fichiers	32
A.1.1	Fichiers à la racine	32
A.1.2	Documentation par dossier	32
A.2	Commandes essentielles	32
A.2.1	Démarrage rapide	32
A.2.2	Pipeline ML complet	33
A.2.3	Tests et qualité	33
A.2.4	Variables d’environnement	34
A.3	Glossaire	34
A.4	Licence et avertissements	35

Introduction

1.1 Contexte médical

Les tumeurs cérébrales représentent un défi diagnostique majeur en neurologie. Selon les statistiques mondiales, environ 308 000 nouveaux cas sont diagnostiqués chaque année. Les quatre classes principales que ce projet adresse sont :

Table 1.1 – Les quatre classes ciblées par le modèle

Classe	Description	Prévalence
Glioma	Tumeur maligne provenant des cellules gliales (haute gravité)	~3 / 100 000 / an
« Méningiome	Tumeur généralement bénigne issue des méninges	~7.8 / 100 000
Hypophysaire	Adénome de la glande hypophyse (souvent bénin)	~77 / 100 000
Aucune tumeur	Cerveau dans les limites normales	—

1.2 Problématique

Le diagnostic manuel par radiologue prend du temps et reste subjectif. Un système d'aide à la décision basé sur le **deep learning** peut :

- Pré-classer rapidement les scans en quelques millisecondes
- Fournir une **carte d'attention Grad-CAM** pour expliquer la décision
- Servir comme deuxième avis pour les radiologues
- Faciliter l'enseignement et la recherche

1.3 Objectifs du projet

1. Construire un classifieur 4-classes performant ($> 95\%$ accuracy)
2. Garantir l'**interprétabilité** via Grad-CAM
3. Déployer une **API REST production-ready** avec sécurité
4. Fournir une **interface web moderne** sans framework lourd
5. Assurer la **reproductibilité** (Docker, CI/CD, tests)
6. Documenter exhaustivement chaque composant

1.4 Stack technique global

Table 1.2 – Technologies utilisées

!10 Catégorie	Outils
Deep Learning	PyTorch 2.11 · torchvision · CUDA 11.8 (optionnel)
Backend API	FastAPI · uvicorn · slowapi · python-dotenv
Frontend	Vanilla JS ES2020 · Chart.js 4.4 · Lucide · jsPDF
UI alternative	Streamlit 1.25+
Données	NumPy · Pandas · OpenCV · Pillow · Albumentations
Visualisation	Matplotlib · Seaborn · Scikit-learn
Tests	pytest · pytest-cov · httpx (TestClient)
Lint / Format	ruff · black · mypy · pre-commit
Container	Docker multi-stage · docker compose · Nginx
CI/CD	GitHub Actions

Architecture globale du projet

2.1 Vue d'ensemble

Le projet est organisé en **16 dossiers top-level** avec une séparation claire des responsabilités. Chaque dossier dispose d'un `README.md` (description détaillée) et d'un `FLOW.md` (arbre de décision visuel des étapes).

```
mri_brain_tumor-main/
LANCEURS
start.py           # Lanceur cross-platform
start.bat / stop.bat # Windows

CODE APPLICATIF
api/               # FastAPI port 8000
interface/         # Frontend Vanilla JS port 3000
streamlit_app/     # Streamlit port 8501
src/               # Modules PyTorch
scripts/           # CLI entry points

ANALYSES & TESTS
dataset_stats/     # 22 analyses auto-découvertes
tests/             # 17 tests pytest

DONNÉES
data/              # raw → processed → splits
models/            # Checkpoints .pth
logs/              # Training history CSV
results/           # Eval outputs

INFRASTRUCTURE
docs/              # Documentation par phase
legacy/            # Code TF archivé
.github/           # CI/CD GitHub Actions
Dockerfile + compose # Containerisation
pyproject.toml     # Config black/ruff/pytest
.env.example       # Config runtime
```

Figure 2.1 – Arborescence du projet (vue synthétique)

2.2 Pipeline complet en 6 phases

Les 6 phases du projet

1. **Phase 1 — Préparation des données** : Kaggle → resize → normalize → split
2. **Phase 2 — Entraînement** : ResNet50 + transfer learning 2 stages
3. **Phase 3 — Évaluation** : Test set 1 054 images → métriques + figures
4. **Phase 4 — Statistiques modulaires** : 22 analyses auto-découvertes
5. **Phase 5 — Déploiement** : API + frontend + Docker
6. **Phase 6 — Qualité** : Tests, CI/CD, pre-commit

2.3 Principes de design

Le projet repose sur cinq principes fondateurs :

Modularité Chaque dossier a une responsabilité unique. `src/` contient uniquement les modules réutilisables. `scripts/` contient uniquement des entry points CLI. `api/` contient uniquement le serveur FastAPI.

Auto-découverte Le dossier `dataset_stats/analyses/` utilise un **registry pattern** : déposer un nouveau fichier `figXX_*.py` le rend automatiquement disponible via le CLI, sans modifier aucun fichier central.

Reproductibilité Tous les paramètres sont centralisés (`config.json`, `.env`, `pyproject.toml`). Les seeds sont fixées (`RANDOM_SEED=42`). Le pipeline est idempotent.

Configurabilité Les variables runtime passent par `.env` (CORS, API key, rate limit, log level). Aucun secret en dur.

Documentation par dossier 17 READMEs + 17 FLOW.md couvrent l'intégralité du projet, avec liens croisés.

Phase 1 : Préparation des données

3.1 Source du dataset

Le projet utilise le dataset Kaggle public **Brain Tumor Classification MRI** de Sartaj Bhuvaaji, contenant **7 023 images** d'IRM cérébrales en JPEG/PNG, classées en 4 catégories.

Table 3.1 – Distribution du dataset Kaggle

Classe	Training	Testing
Glioma	1 321	300
Meningioma	1 339	306
No Tumor	1 595	405
Pituitary	1 457	300
Total	5 712	1 311

3.2 Pipeline de prétraitement

Le module `src/data_pipeline.py` orchestre l'ensemble du prétraitement avec ces étapes :

```

1. Validation      (PIL.open -> detecte fichiers corrompus)
2. Conversion RGB  (grayscale -> 3 channels, RGBA      drop alpha)
3. Resize 256      (maintient aspect ratio + padding noir)
4. CenterCrop 224  (recadrage centré 224x224)
5. Normalize       (uint8[0,255]      float32[0,1] + ImageNet stats)
6. Save .npz       (binaire numpy, ~4x plus rapide que JPEG)
```

Listing 3.1 – Étapes du prétraitement

Pourquoi le format .npz ?

- Chargement **4 fois plus rapide** qu'un JPEG décodé à chaque batch
- Stockage en `float32` déjà normalisé (pas de conversion en runtime)
- Compatible direct avec `torch.from_numpy()`
- Léger reproches : taille ~2 GB total, mais acceptable

3.3 Augmentation des données (Albumentations)

L'augmentation est appliquée **uniquement au train set**, à la volée dans le `DataLoader`. Le pipeline utilise la bibliothèque Albumentations pour ses performances supérieures à `torchvision.transforms` :

```

1 train_transform = A.Compose([
2     A.HorizontalFlip(p=0.5),
3     A.RandomRotate90(p=0.3),
4     A.Rotate(limit=15, p=0.4),           # rotation -15 / +15
5     A.RandomResizedCrop(scale=(0.85, 1.0)), # zoom in/out léger
6     A.RandomBrightnessContrast(0.2, p=0.4), # luminosité +/- 20%
7     A.GaussNoise(var_limit=(10, 50), p=0.2), # bruit Gaussien
8     A.ElasticTransform(p=0.2),           # deformation organique
9     A.Normalize([0.485, 0.456, 0.406],
10                [0.229, 0.224, 0.225]),
11     ToTensorV2(),
12 ])

```

Listing 3.2 – Pipeline d’augmentation Albumentations

Le val/test set ne reçoit que la `Normalize` pour conserver une évaluation déterministe.

3.4 Split stratifié

Le module `src/dataset_split.py` sépare les images en train/val/test en **préservant la distribution des classes** grâce à `sklearn.model_selection.StratifiedShuffleSplit` :

Table 3.2 – Splits finaux (seed=42, stratifié)

Split	Images	Ratio	Usage
Train	4 915	70%	Apprentissage des poids
Val	1 054	15%	EarlyStopping, ReduceLROnPlateau
Test	1 054	15%	Évaluation finale (jamais vu pendant train)
Total	7 023	100%	

Le résultat est sauvegardé dans `data/splits/split_info.json` (632 KB) qui sert de **source de vérité unique** pour tous les autres modules (training, evaluation, dataset_stats).

Phase 2 : Entraînement du modèle

4.1 Architecture du modèle

Le modèle est défini dans `src/model_architecture.py` et combine deux éléments :

1. Un **backbone ResNet50** pré-entraîné sur ImageNet (transfer learning)
2. Une **tête de classification custom** adaptée à 4 classes

```

Input: RGB tensor (B, 3, 224, 224)
|
v
+-----+
| ResNet50 backbone |
| - conv1 -> bn1 -> relu -> maxpool |
| - layer1 (3x Bottleneck, 256) |
| - layer2 (4x Bottleneck, 512) |
| - layer3 (6x Bottleneck, 1024) |
| - layer4 (3x Bottleneck, 2048) <- hooks Grad-CAM
| - avgpool -> flatten |
| base_model.fc = Identity() |
+-----+
| (B, 2048)
v
+-----+
| Custom classifier head |
| Linear(2048 -> 512) |
| ReLU |
| BatchNorm1d(512) |
| Dropout(0.4) |
| Linear(512 -> 4) |
+-----+
| (B, 4)
v
Logits -> Softmax -> 4 classes

```

Figure 4.1 – Architecture ResNet50Classifier

Table 4.1 – Distribution des paramètres

Couches	Params	Stage 1	Stage 2
ResNet50 backbone	~23.5M	frozen	trainable
Tête custom	~1.05M	trainable	trainable
Total	~24.6M	~1.05M	~15M

4.2 Stratégie de transfer learning en 2 stages

Le `PyTorchTrainer` (dans `src/model_trainer.py`) implémente une stratégie en deux temps qui surpasse l'entraînement classique :

Stage 1 — Adaptation rapide de la tête

- **Durée** : 20 époques
- **Learning rate** : 1×10^{-3} (élevé pour faire bouger rapidement la tête)
- **Backbone** : entièrement gelé (`freeze_base_layers()`)
- **Trainable params** : $\sim 1\text{M}$ (seulement la tête)
- **Goal** : adapter rapidement les features ImageNet à la tâche médicale

Stage 2 — Fine-tuning précis

- **Durée** : 30 époques
- **Learning rate** : 1×10^{-4} ($10\times$ plus petit)
- **Backbone** : 30 dernières couches dégelées (`unfreeze_base_layers(num_layers=30)`)
- **Trainable params** : $\sim 15\text{M}$
- **Goal** : affiner subtilement les features bas-niveau pour la spécificité médicale

4.2.1 Pourquoi 2 stages ?

Table 4.2 – Comparaison entraînement classique vs 2 stages

Sans 2 stages	Avec 2 stages
LR trop élevé sur ResNet → catastrophic forgetting	Stage 1 stabilise la tête sans toucher au backbone
Convergence en ~ 50 époques avec instabilités	Convergence en $\sim 30-40$ époques avec courbes lisses
Risque d'overfitting tête vs backbone	Découplage clair des phases

4.3 Callbacks et optimisations

- **EarlyStopping** (`patience=7`) : arrête si `val_loss` ne décroît plus
- **ReduceLROnPlateau** (`factor=0.5`, `patience=3`) : divise le LR par 2 si stagnation
- **ModelCheckpoint** : sauvegarde le meilleur modèle (sur `val_loss`)
- **TensorBoard** : log par batch pour monitoring fin
- **Class weights** : compensation automatique de l'imbalance des classes

4.4 Hyperparamètres complets

Table 4.3 – Configuration par défaut (`get_default_config()`)

Paramètre	Valeur
<code>num_classes</code>	4
<code>image_size</code>	224
<code>batch_size</code>	24
<code>num_workers</code>	0 (Windows-friendly)
<code>dense_units</code>	512
<code>dropout_rate</code>	0.4
<code>learning_rate_s1</code>	1×10^{-3}
<code>learning_rate_s2</code>	1×10^{-4}
<code>epochs_stage1</code>	20
<code>epochs_stage2</code>	30
<code>weight_decay</code>	1×10^{-4}
<code>optimizer</code>	Adam
<code>loss</code>	CrossEntropyLoss + class_weights
<code>early_stopping_patience</code>	7
<code>lr_reduce_factor</code>	0.5
<code>lr_reduce_patience</code>	3

4.5 Durée d'entraînement

- **CPU (Intel i5/i7)** : 4–6 heures pour 50 époques
- **GPU NVIDIA RTX 4060 (8 GB VRAM)** : 10-12 minutes
- **Mémoire VRAM** : ~ 3.2 GB occupés avec `batch_size=24`

Phase 3 : Évaluation

5.1 Procédure d'évaluation

Le script `scripts/evaluate.py` charge le dernier checkpoint dans `models/` et l'évalue sur les **1 054 images de test** (jamais vues pendant l'entraînement).

```

1 model = ResNet50Classifier(num_classes=4, pretrained=False)
2 model.load_state_dict(checkpoint["model_state_dict"])
3 model.to(device).eval()
4
5 with torch.no_grad():
6     for x, y in test_loader:
7         outputs = model(x.to(device))
8         probs = F.softmax(outputs, dim=1)
9         preds = torch.argmax(probs, dim=1)
10        # accumule y_true, y_pred, y_proba

```

Listing 5.1 – Pipeline d'évaluation simplifié

5.2 Résultats globaux

Performance finale

98.96%

1 043 / 1 054 images correctement classifiées

5.3 Métriques par classe

Table 5.1 – Classification report sur le test set

navy!10 Classe	Precision	Recall	F1-score	Support
Glioma	0.99	0.99	0.99	260
« Meningioma	0.98	0.98	0.98	257
No Tumor	1.00	1.00	1.00	273
Pituitary	0.99	0.99	0.99	264
Macro avg	0.99	0.99	0.99	1 054
Weighted avg	0.99	0.99	0.99	1 054

5.4 Matrice de confusion

Table 5.2 – Matrice de confusion (true vs predicted)

	Glioma	Meningioma	No Tumor	Pituitary
Glioma	257	2	0	1
Meningioma	3	252	0	2
No Tumor	0	1	272	0
Pituitary	1	1	0	262

Observations :

- La classe **No Tumor** est presque parfaitement détectée (272/273)
- Les confusions principales sont **Glioma** ↔ « Meningioma (5 erreurs au total) — c’est cohérent car ces deux types peuvent visuellement se ressembler en IRM
- Aucune confusion grave entre tumeur et non-tumeur

5.5 Performances temporelles

Table 5.3 – Latence d’inférence selon le device

Device	Latence/image	Throughput
GPU NVIDIA RTX 4060	~2 ms	~5 img/s
CPU Intel i7 (8 cores)	~36 ms	~28 img/s

5.6 Sorties d’évaluation

L’évaluation produit dans `results/` :

- `evaluation_summary.txt` : métriques globales formatées
- `test_results.csv` : prédictions par image (1 054 lignes)
- `confusion_matrix.png` : heatmap seaborn
- `class_performance.png` : bar chart P/R/F1
- `training_history.png` : courbes loss/accuracy (via `visualize_training.py`)

Phase 4 : Statistiques modulaires (22 analyses)

6.1 Architecture par registre

Le dossier `dataset_stats/` implémente une **architecture auto-adaptative** où chaque analyse est un fichier autonome auto-découvert. Aucun fichier central ne nécessite de modification pour ajouter une 23^e analyse.

Principe : 1 fichier

Déposer un fichier `analyses/figXX*.py` respectant le contrat (NAME, TITLE, CATEGORY, REQUIRES, ORDER, `run()`) le rend immédiatement disponible via le CLI `python dataset_stats/run.py -list`.

6.2 Structure du package

```
dataset_stats/
run.py          # CLI : -list -only -except -filter -category

core/           # Infrastructure partagée
config.py       # Constantes (paths, classes, couleurs)
data.py         # Helpers I/O (load_npy, list_class_files...)
plotting.py     # matplotlib + save_figure (par catégorie)
registry.py     # Auto-découverte

analyses/       # 22 modules autonomes
fig01_class_distribution.py
fig02_split_distribution.py
...
fig22_class_focus_confusion.py

outputs/        # Généré (auto-organisé par catégorie)
figures/{overview,image,training,evaluation,errors}/
data/{idem}/
summary.json
```

Figure 6.1 – Architecture du package `dataset_stats/`

6.3 Catalogue des 22 analyses

6.3.1 Vue d'ensemble (overview) — 2 analyses

#	Analyse	Description
01	<code>class_distribution</code>	Bar + pie chart par classe
02	<code>split_distribution</code>	Stratified train/val/test 70/15/15

6.3.2 Propriétés des images (image) — 9 analyses

#	Analyse	Description
03	pixel_statistics	Mean / std / min / max par classe
04	image_properties	Carte info (sizes, channels, dtype)
05	sample_grid	Grille 5 exemples par classe
06	augmentation_examples	Original vs 7 transformations
10	intensity_histograms	Distribution intensité (overlay+KDE)
17	mean_image	Image moyenne pixel-wise par classe
19	channel_stats	Mean/std par canal R/G/B
20	image_entropy	Distribution Shannon entropy
21	class_similarity	Matrice cosine inter-classes

6.3.3 Entraînement (training) — 2 analyses

#	Analyse	Description
07	training_history	Courbes loss/acc 50 époques (Stage 1+2)
13	lr_schedule	Visualisation du LR scheduler

6.3.4 Évaluation (evaluation) — 5 analyses

#	Analyse	Description
08	confusion_matrix	Heatmap counts + recall normalisé
09	per_class_metrics	Precision/Recall/F1 par classe
14	roc_curves	ROC one-vs-rest + AUC par classe
15	pr_curves	Precision-Recall + Average Precision
16	calibration	Reliability diagram + Brier score

6.3.5 Analyse des erreurs (errors) — 4 analyses

#	Analyse	Description
11	confidence_distribution	Confiance correct vs erreur (boxplot)
12	misclassification	Top patterns d'erreur + error rate
18	top_predictions	Top-1 vs Top-2 vs Top-3 accuracy
22	class_focus_confusion	4 mini-matrices "1 vs rest"

6.4 Mécanisme d'auto-découverte

Le cœur du système est dans `core/registry.py` :

```
def discover() -> list[RegisteredAnalysis]:
    items = []
    for py_file in glob("analyses/fig*.py"):
        mod = importlib.import_module(f"analyses.{py_file.stem}")
        items.append(RegisteredAnalysis(
            name=mod.NAME,
```

```

7         title=mod.TITLE,
8         category=mod.CATEGORY,
9         requires=mod.REQUIRES,
10        order=mod.ORDER,
11        run=mod.run,
12    ))
13    return sorted(items, key=lambda x: x.order)

```

Listing 6.1 – Pseudo-code du registry

6.5 Organisation automatique des sorties

Une astuce élégante : le module `plotting.py` maintient un **contexte global** de catégorie qui est posé/libéré par le registry autour de chaque `run()` :

```

1 # core/plotting.py
2 _CURRENT_CATEGORY = None
3
4 def set_current_category(cat):
5     global _CURRENT_CATEGORY
6     _CURRENT_CATEGORY = cat
7
8 def save_figure(fig, name, *, category=None):
9     cat = category or _CURRENT_CATEGORY
10    out = FIGURES_DIR / cat if cat else FIGURES_DIR
11    out.mkdir(parents=True, exist_ok=True)
12    fig.savefig(out / f"{name}.png")
13
14 # core/registry.py - dans run_one()
15 set_current_category(item.category)
16 try:
17     data = item.run()
18 finally:
19     set_current_category(None)

```

Listing 6.2 – Context manager de catégorie

Résultat : aucune analyse ne touche au contexte. La magie est centralisée et transparente. Les figures sont automatiquement rangées dans `outputs/figures/<category>/`.

6.6 CLI flexible

```

1 # Lister les 22 analyses (groupées par catégorie)
2 python dataset_stats/run.py --list
3
4 # Tout exécuter
5 python dataset_stats/run.py
6
7 # Sélection précise par ID

```

```
8 python dataset_stats/run.py --only 01,03,07,14,15
9
10 # Par categorie
11 python dataset_stats/run.py --category evaluation
12
13 # Recherche par mot-cle
14 python dataset_stats/run.py --filter training
15
16 # Tout sauf certaines
17 python dataset_stats/run.py --except 06,10
```

Listing 6.3 – Exemples d'utilisation

Phase 5 : Déploiement

7.1 API FastAPI

7.1.1 Architecture

L'API est dans `api/app.py` et expose 7 endpoints REST. Le modèle ResNet50 est chargé **une seule fois** au démarrage en singleton.

```

1 @asynccontextmanager
2 async def lifespan(app: FastAPI):
3     global predictor
4     log.info("Chargement du modele...")
5     try:
6         predictor = load_model() # singleton
7         log.info("Modele pret.")
8     except Exception as exc:
9         log.error(f"Erreur chargement modele : {exc}")
10        predictor = None
11    yield
12    log.info("Arret de l'API.")

```

Listing 7.1 – Lifecycle de l'API

7.1.2 Les 7 endpoints

Table 7.1 – Endpoints REST exposés

#	Route	Auth	Description
1	GET /health	✗	Statut + métadonnées modèle
2	POST /predict	✓	Image → classe + Grad-CAM (cache MD5)
3	GET /random	✗	Image dataset aléatoire + prédiction
4	GET /explain/{class}	✗	Infos médicales (description, urgence)
5	GET /sample/{class}	✗	Image d'exemple base64
6	POST /batch	✓	Jusqu'à 20 images en parallèle
7	GET /stats	✗	Statistiques de session

7.1.3 Pipeline d'une requête POST /predict

1. Validation MIME (JPEG/PNG) + taille (≤ 10 MB)
2. Calcul MD5 du contenu → cache lookup (LRU 100 entrées)
3. Si cache hit : retour instantané avec `from_cache: true`

4. Sinon : preprocessing (Resize 224 + Normalize ImageNet)
5. Forward pass ResNet50 (~30 ms CPU / 10 ms GPU)
6. Softmax + argmax → classe + probabilités
7. Grad-CAM via hooks PyTorch sur `layer4[-1]` (~80 ms CPU)
8. Encodage base64 (image + heatmap + overlay)
9. Update session_stats + cache write
10. Retour JSON (~150 ms total CPU)

7.1.4 Grad-CAM en pur PyTorch

Le projet implémente Grad-CAM **sans dépendance TensorFlow**, directement avec les hooks PyTorch :

```

1 def compute_gradcam(model, tensor, target_idx):
2     activations, gradients = [], []
3     fwd = model.base_model.layer4[-1].register_forward_hook(
4         lambda m, i, o: activations.append(o)
5     )
6     bwd = model.base_model.layer4[-1].register_full_backward_hook(
7         lambda m, gi, go: gradients.append(go[0])
8     )
9     img = tensor.clone().requires_grad_(True)
10    with torch.enable_grad():
11        out = model(img)
12        one_hot = torch.zeros_like(out)
13        one_hot[0][target_idx] = 1.0
14        out.backward(gradient=one_hot)
15    fwd.remove(); bwd.remove()
16
17    acts = activations[0].squeeze(0)
18    grads = gradients[0].squeeze(0)
19    weights = grads.mean(dim=(1, 2))
20    cam = (weights[:, None, None] * acts).sum(dim=0)
21    cam = F.relu(cam)
22    cam = (cam - cam.min()) / (cam.max() - cam.min())
23    return cam.cpu().numpy()

```

Listing 7.2 – Grad-CAM via hooks (extrait)

7.1.5 Sécurité

7.2 Frontend Vanilla JavaScript

7.2.1 Stack technique

L'interface dans `interface/` est construite **sans framework** :

Table 7.2 – Couches de sécurité de l'API

Mesure	Outil / config
CORS configurable	BRAINSCAN_CORS_ORIGINS (CSV)
Rate limiting	slowapi 60 req/min/IP par défaut
Authentification	Header X-API-Key (optionnel)
Validation MIME	whitelist JPEG/PNG
Limite taille	10 MB par fichier
Cache LRU	100 entrées max (anti-DOS)

- **HTML5** sémantique (~388 lignes)
- **CSS3** avec custom properties pour theming light/dark (~922 lignes)
- **Vanilla JS ES2020** en 16 blocs commentés (~1 350 lignes)
- CDN externes : Chart.js 4.4, Lucide, jsPDF, Inter font
- **Aucun** node_modules/, **aucun** build step

7.2.2 Architecture en 16 blocs

Table 7.3 – Découpage de app.js

Bloc	Rôle
1	CONFIG, CLASS_STYLES, AppState, activeFilters, DOM cache
2	API object (6 fetch wrappers async)
3	Toasts (max 3 simultanés, auto-dismiss)
4	API status polling (toutes les 30s)
5	File handling + heuristique qualité IRM (canvas)
6	Loader + clearUpload
7	displayResults (badge, confiance, accordion)
8	Chart.js horizontal bars + plugin labels
9	History (compress thumbnails, save, load)
10	Filters + pagination + render
11	Session stats + CSV export
12	Zoom modal + dark mode + opacity slider
13	Export PNG (canvas) + Export PDF (jsPDF)
14	Compare mode (2 IRM en parallèle)
15	Batch analysis (jusqu'à 20 fichiers)
16	Listeners + DOMContentLoaded

7.2.3 Fonctionnalités utilisateur

- Upload **drag-and-drop** ou click-to-browse
- Détection automatique **IRM-like** (heuristique canvas 64×64)
- Affichage **Grad-CAM** : original / heatmap / overlay (zoomable)

- Graphique horizontal **Chart.js** animé
- Accordion **infos médicales** (via `/explain`)
- **Historique** persisté en `localStorage` (max 100, filtres + pagination)
- Export **PDF** (jsPDF), **PNG** (canvas), **CSV** (historique)
- **Mode comparaison** (2 IRM en parallèle via `Promise.all`)
- **Batch** (jusqu'à 20 images séquentielles)
- **Dark/Light mode** avec persistance

7.3 Frontend alternatif : Streamlit

Le dossier `streamlit_app/app.py` fournit une **alternative mono-fichier** qui charge directement le modèle (sans API séparée) :

Table 7.4 – Streamlit vs FastAPI + Vanilla JS

Critère	Streamlit	FastAPI + JS
Démarrage	1 commande	2 services
Architecture	Monolithique	REST API découplée
Multi-utilisateurs	Non	Oui
Personnalisation UI	Limitée	Totale
Idéal pour	Demos rapides	Production

7.4 Containerisation Docker

7.4.1 Dockerfile multi-stage

Le `Dockerfile` utilise une approche **multi-stage** pour minimiser la taille finale :

```

1 # Stage 1 (builder) : installe deps avec gcc
2 FROM python:3.11-slim AS builder
3 RUN apt-get install build-essential libgl1
4 RUN pip install --user -r requirements.txt
5
6 # Stage 2 (runtime) : copie packages, sans compilateurs
7 FROM python:3.11-slim AS runtime
8 COPY --from=builder /root/.local /root/.local
9 RUN apt-get install libgl1 curl      # libs runtime uniquement
10 USER brainscan                     # non-root
11 HEALTHCHECK CMD curl -f http://localhost:8000/health
12 CMD ["uvicorn", "api.app:app", "--host", "0.0.0.0", "--port", "8000"]

```

Listing 7.3 – Stages du Dockerfile

Caractéristiques :

- Image finale ~1.5 GB
- Tourne en utilisateur non-root (**brainscan**)
- Healthcheck intégré
- OpenCV headless (**libgl1** sans interface graphique)

7.4.2 docker-compose

Le `docker-compose.yml` orchestre l'API et un Nginx servant l'interface statique :

```
1 # Build + lance API (8000) + Nginx (3000)
2 docker compose up --build
3
4 # En arriere-plan
5 docker compose up -d
6
7 # Stop propre
8 docker compose down
```

Listing 7.4 – Lancement Docker

Phase 6 : Qualité du code

8.1 Suite de tests pytest

Le dossier `tests/` contient 17 tests couvrant l'API et le registry `dataset_stats/`.

Table 8.1 – Couverture des tests

Fichier	Tests	Couverture
<code>test_api.py</code>	13	E2E tous les endpoints (TestClient in-process)
<code>test_dataset_stats.py</code>	4	Registry auto-discovery + validation
Total	17	Passing en ~10 secondes

8.1.1 Détail des 13 tests API

1. `test_health_ok` : `/health` renvoie les bonnes métadonnées
2. `test_predict_jpeg` : prédiction sur JPEG aléatoire
3. `test_predict_png` : prédiction sur PNG aléatoire
4. `test_predict_invalid_format` : 400 sur `text/plain`
5. `test_predict_cache_hit` : 2 mêmes uploads → `from_cache: true`
6. `test_random_returns_prediction` : `/random` avec `true_class`
7. `test_explain_glioma` : infos médicales valides
8. `test_explain_no_tumor` : idem
9. `test_explain_unknown_class` : 404 sur classe inconnue
10. `test_sample_pituitary` : image base64 retournée
11. `test_sample_unknown_class` : 404
12. `test_stats` : structure de réponse complète
13. `test_batch` : 2 images analysées en lot

8.1.2 Fixtures partagées (`conftest.py`)

```

1 @pytest.fixture(scope="session")
2 def client():
3     """TestClient FastAPI in-process (model load 1 fois)."""
4     from fastapi.testclient import TestClient

```

```

5     from api.app import app
6     with TestClient(app) as c:
7         yield c
8
9 @pytest.fixture
10 def fake_jpeg_bytes() -> bytes:
11     """JPEG aleatoire 224x224 RGB en memoire."""
12     arr = (np.random.rand(224, 224, 3) * 255).astype("uint8")
13     img = Image.fromarray(arr)
14     buf = io.BytesIO()
15     img.save(buf, format="JPEG")
16     return buf.getvalue()

```

Listing 8.1 – Fixtures pytest

8.2 CI/CD GitHub Actions

Le pipeline `.github/workflows/ci.yml` se déclenche sur `push` et `pull_request` avec 3 jobs séquentiels :

Table 8.2 – Pipeline CI/CD

#	Job	Outils	Durée
1	lint	ruff + black -check	~30 s
2	test (needs lint)	compileall + pytest	~2 min
3	docker (needs test)	Buildx + cache GHA	~1-3 min

8.3 Pre-commit hooks

Le fichier `.pre-commit-config.yaml` active des hooks Git locaux pour intercepter les problèmes avant le push :

```

1 - trailing-whitespace
2 - end-of-file-fixer
3 - check-yaml / check-json
4 - check-added-large-files (max 500 KB)
5 - detect-private-key
6 - ruff --fix
7 - black

```

Listing 8.2 – Hooks pre-commit configurés

8.4 Configuration centralisée (pyproject.toml)

Toute la config tooling est dans un seul fichier :

```

1 [tool.black]
2 line-length = 100
3 target-version = ["py311"]

```

```
4  
5 [tool.ruff]  
6 line-length = 100  
7 select = ["E", "W", "F", "I", "B", "UP"]  
8 ignore = ["E501", "B008"]  
9  
10 [tool.pytest.ini_options]  
11 testpaths = ["tests"]  
12 python_files = "test_*.py"  
13 addopts = "-v --tb=short"
```

Listing 8.3 – Extrait de pyproject.toml

Résultats et benchmarks

9.1 Synthèse des performances

Table 9.1 – Tableau de bord global

Métrique		Valeur
Test accuracy globale		98.96%
Erreurs de classification (test)		11 / 1 054
Macro F1-score		0.99
Inférence CPU (i7)		~36 ms/image
Inférence GPU (RTX 2050)		~10 ms/image
lightgray	Code base	
Lignes Python (modules + scripts)		~3 700
Lignes Frontend (HTML+CSS+JS)		~2 660
Lignes documentation (READMEs+FLOW)		~6 400
lightgray	Tests & qualité	
Tests pytest		17 / 17 passing
Couverture endpoints API		100%
Pipeline CI durée totale		3-5 min
lightgray	Architecture modulaire	
Analyses statistiques		22 (auto-découvertes)
Endpoints API REST		7
Frontends disponibles		2 (Vanilla JS + Streamlit)
lightgray	Modèle	
Architecture		ResNet50 + tête custom
Paramètres totaux		~24.6M
Taille checkpoint		~95 MB

9.2 Comparaison avant/après refactor

Le projet a subi un **refactor majeur** qui a dramatiquement amélioré la qualité :

Table 9.2 – Métriques avant/après refactor

Aspect	Avant	Après
Code mort TF dans <code>src/</code>	6 fichiers	0 (déplacés ou supprimés)
Doublons <code>dataset_stats/</code>	3 (<code>figures/</code> , <code>generate_stats.py</code> , <code>summary.json</code>)	0
Tests automatisés	0	17
Logger	<code>print()</code> partout	Logger Python configurable
Sécurité API	CORS <code>["*"]</code> hardcodé	Config + API key + rate-limit
Docker	✗	Multi-stage + compose
CI/CD	✗	GitHub Actions (lint+test+build)
Pre-commit	✗	✓
Git LFS config	✗	✓
<code>.env</code> support	✗	Avec template
Documentation	4 READMEs	17 READMEs + 17 FLOW.md

Conclusion et perspectives

10.1 Bilan

BrainScan AI est un projet **abouti et production-ready** qui démontre l'application de techniques modernes de deep learning à un problème médical concret. Les choix architecturaux (transfer learning 2 stages, registry auto-découverte, séparation API/frontend) ont permis d'atteindre des résultats remarquables :

- **98.96% d'accuracy** sur un test set indépendant
- **Latence sub-100ms** en inférence
- **Interprétabilité** via Grad-CAM en pur PyTorch
- **Qualité de code** : 17 tests, CI verte, pre-commit, Docker
- **Documentation exhaustive** : 17 READMEs + 17 FLOW.md

10.2 Limites actuelles

Limites à connaître

- Le modèle a été entraîné sur un **seul dataset Kaggle** : généralisation à d'autres cohortes non vérifiée
- Pas de **cross-validation** K-fold (split unique 70/15/15)
- Pas d'**ensemble** de modèles (un seul ResNet50)
- L'usage est strictement **éducatif** : ne remplace pas un radiologue
- Le modèle ne distingue pas le **grade** de la tumeur (Glioma I-IV par exemple)
- Pas d'**intégration EHR** ou DICOM (uniquement JPEG/PNG)

10.3 Pistes d'amélioration

10.3.1 Performance modèle

- **Ensemble** : moyenner ResNet50 + EfficientNet + ViT
- **Test-Time Augmentation (TTA)** : moyenner les prédictions sur 8 augmentations
- **Cross-validation 5-fold** pour robustesse
- Datasets supplémentaires : BraTS, TCGA-GBM

10.3.2 Production

- **Quantization** (INT8) : modèle 4× plus léger (~25 MB)
- **ONNX export** pour inférence cross-platform
- **Monitoring** : Sentry pour erreurs, Prometheus pour métriques
- **Cache distribué** : Redis au lieu du LRU in-memory
- **Async loading** du modèle au boot pour API instantanée

10.3.3 Fonctionnalités

- Support **DICOM** (format standard en radiologie)
- Détection multi-tumeurs (segmentation au lieu de classification)
- Module de **rapport médical** structuré (PDF auto-généré)
- **API key management** avec dashboard admin
- Internationalisation (i18n) pour le frontend

10.4 Mot final

Ce projet illustre qu'il est possible de construire un système ML médical **de bout en bout** en respectant les meilleures pratiques modernes : modularité, tests, documentation, sécurité, reproductibilité. La courbe d'apprentissage est élevée mais le résultat est durable, maintenable et étendu de manière naturelle (ajouter une 23^e analyse statistique prend 5 minutes grâce à l'auto-découverte).

Le projet en chiffres

17	READMEs détaillés
17	FLOW.md visuels
22	Analyses modulaires
17	Tests automatisés
7	Endpoints REST
2	Frontends (FastAPI+JS et Streamlit)
4	Classes médicales
1	Architecture solide

Annexes

A.1 Inventaire complet des fichiers

A.1.1 Fichiers à la racine

Fichier	Rôle
start.py	Lanceur intelligent cross-platform
start.bat	Lanceur Windows (double-clic)
stop.bat	Arrête les serveurs (ports 8000/3000)
Dockerfile	Image Docker multi-stage
docker-compose.yml	Orchestration API + Nginx
requirements.txt	Dépendances production
requirements-dev.txt	Dépendances dev (pytest, ruff, black)
runtime.txt	Python 3.11.9 (pour Render)
config.json	Métadonnées projet
pyproject.toml	Config black + ruff + pytest
pytest.ini	Config pytest
.env.example	Template variables d'environnement
.gitignore	Patterns à exclure du repo
.gitattributes	Git LFS pour *.pth
.dockerignore	Patterns à exclure du build Docker
.pre-commit-config.yaml	Hooks pre-commit
README.md	Documentation principale (510 lignes)
FLOW.md	Pipeline global du projet

A.1.2 Documentation par dossier

Les 17 dossiers du projet ont chacun deux fichiers de documentation :

A.2 Commandes essentielles

A.2.1 Démarrage rapide

```

1 # 1. Cloner et installer
2 git clone https://github.com/iliass06/Brain-Tumor-project.git
3 cd Brain-Tumor-project
4 python -m venv venv
5 venv\Scripts\pip install -r requirements.txt
6
7 # 2. Lancer (cross-platform)
8 python start.py

```

Table A.1 – Documentation par dossier

Dossier	README	FLOW	Sujet principal
api/	✓	✓	FastAPI · 7 endpoints
interface/	✓	✓	Vanilla JS · 16 blocs
streamlit_app/	✓	✓	Alternative Streamlit
src/	✓	✓	Modules PyTorch
scripts/	✓	✓	Entry points CLI
tests/	✓	✓	17 tests pytest
legacy/	✓	✓	Code TF archivé
dataset_stats/	✓	✓	22 analyses
dataset_stats/core/	✓	✓	Infrastructure
dataset_stats/analyses/	✓	✓	Contrat des analyses
data/	✓	✓	Dataset Kaggle
models/	✓	✓	Checkpoints .pth
logs/	✓	✓	Training history
results/	✓	✓	Eval outputs
docs/	✓	✓	Phases historiques
.github/	✓	✓	CI/CD

```

9
10 # OU (Windows simple)
11 double-clic sur start.bat
12
13 # OU (Docker)
14 docker compose up --build

```

Listing A.1 – Premier lancement

A.2.2 Pipeline ML complet

```

1 # Phase 1 : Preparation des donnees
2 python -m src.data_pipeline
3
4 # Phase 2 : Entraînement
5 python scripts/train.py
6
7 # Phase 3 : Evaluation
8 python scripts/evaluate.py
9
10 # Phase 4 : Statistiques (22 analyses)
11 python dataset_stats/run.py
12
13 # Verification courbes
14 python scripts/visualize_training.py

```

Listing A.2 – Pipeline projet de bout en bout

A.2.3 Tests et qualité

```

1 # Tests pytest (17 tests, ~10 secondes)
2 venv\Scripts\python.exe -m pytest tests/ -v
3
4 # Avec couverture
5 pytest --cov=api --cov=src tests/
6
7 # Lint local (avant push)
8 ruff check api/ src/ scripts/
9 black --check api/ src/ scripts/
10
11 # Auto-format
12 black api/ src/ scripts/
13
14 # Pre-commit (1x install)
15 pre-commit install

```

Listing A.3 – Tests et lint

A.2.4 Variables d’environnement

Table A.2 – Variables .env

Variable	Défaut	Effet
BRAINSCAN_MODEL_PATH	models/final_model_*.pth	Chemin checkpoint
BRAINSCAN_CORS_ORIGINS	*	CSV ou wildcard
BRAINSCAN_API_KEY	(vide)	Auth header X-API-Key
BRAINSCAN_RATE_LIMIT	60	Req/min/IP
BRAINSCAN_DISABLE_RATELIMIT	0	1 = bypass
BRAINSCAN_LOG_LEVEL	INFO	DEBUG, INFO, WARNING, ERROR

A.3 Glossaire

Albumentations Bibliothèque Python de data augmentation pour images, plus rapide que torchvision.transforms.

BatchNorm Couche de normalisation par batch qui stabilise l’entraînement.

CORS Cross-Origin Resource Sharing — mécanisme HTTP qui permet à un site web d’autoriser des requêtes depuis d’autres origines.

Dropout Technique de régularisation : éteint aléatoirement un pourcentage de neurones pendant l’entraînement.

EarlyStopping Arrête l’entraînement si la métrique de validation ne s’améliore plus pendant N époques.

F1-score Moyenne harmonique de precision et recall : $F1 = 2 \cdot \frac{P \cdot R}{P + R}$.

FastAPI Framework Python moderne pour construire des APIs REST asynchrones.

Grad-CAM Gradient-weighted Class Activation Mapping — technique d’interprétabilité qui visualise les zones d’une image qui ont influencé la décision du modèle.

LR (Learning Rate) Taux d’apprentissage : pas de descente dans l’optimisation.

ResNet50 Architecture de réseau de neurones convolutif à 50 couches avec connexions résiduelles (skip connections).

Singleton Pattern de conception : une seule instance partagée pour toute la durée de vie de l'application.

SlowAPI Bibliothèque de rate limiting pour FastAPI.

Softmax Fonction qui transforme un vecteur de logits en distribution de probabilités sommant à 1.

Stratification Procédé de split qui préserve la distribution des classes dans chaque sous-ensemble.

TestClient Client HTTP in-process de FastAPI pour les tests (pas besoin d'uvicorn).

Transfer Learning Réutiliser un modèle pré-entraîné sur une grande tâche (ImageNet) pour une tâche cible plus petite.

TTA (Test-Time Augmentation) Moyenne des prédictions sur plusieurs augmentations de la même image en inférence.

A.4 Licence et avertissements

Avertissement médical

Cet outil est destiné à un **usage éducatif et de recherche uniquement**. Il ne remplace en aucun cas un diagnostic médical professionnel. Les performances annoncées (98.96%) ont été obtenues sur un dataset Kaggle public et ne garantissent pas une généralisation à toutes les populations cliniques.

Consulter toujours un professionnel de santé qualifié pour tout sujet médical.

Licence : Projet à usage éducatif.

Dataset : [Brain Tumor Classification MRI](#) de Sartaj Bhuvaji (Kaggle).

Frameworks : PyTorch · FastAPI · Chart.js · jsPDF · Lucide — sous leurs licences respectives.

BrainScan AI

ResNet50 · 98.96% accuracy · 22 analyses · Production-ready

Fin du rapport — 4 mai 2026